

GPUを用いた粒子法シミュレーションのためのスライスデータ構造*

A Sliced Data Structure for Particle-based Simulations on GPUs

原田 隆宏¹, 越塚 誠一², 河口 洋一郎¹

Takahiro HARADA, Seiichi KOSHIZUKA and Yoichiro KAWAGUCHI

¹ 東京大学大学院情報学環 (〒133-0033 東京都文京区本郷 7-3-1)

² 東京大学大学院工学系研究科 (〒133-8656 東京都文京区本郷 7-3-1)

We present a sliced data structure that is effective for neighboring particle search of particle-based simulations. In the method, a grid is constructed dynamically to fit to a particle distribution. Rather than computing the grid fitting perfectly to a particle distribution where there are no voxels which does not store particle indices, we compute a grid with some margin to the distribution. This lowers the computation cost of generating the data structure. Before storing indices of particles to a grid, key values which are used to compute the index of a voxel are computed. The present data structure can be introduced to particle-based simulations on the Graphics Processing Unit (GPU) because the construction of this data structure and access to stored values can be also performed entirely on the GPU. The present data structure free us from a restriction of computation region with a fixed grid and so that we can simulate particle motions in a larger area. Moreover, the cost of the present method is low enough to be able to use for real-time applications. In this paper, we first introduce the sliced data structure and then describe how to implement it on the GPU. Then the present method is applied to particle-based simulations and quantitative evaluations are presented.

Key Words: Particle-based Simulation, Graphics Hardware, Computer Graphics, Data Structure

1. 序論

コンピュータグラフィックスでは流体のシミュレーションはなくてはならないものになっている。それは流体の挙動が複雑であるために、手で作成するのが困難であるからである。この領域では高品質な映像を高速に作成するというニーズがあり、様々な研究が行なわれている。

粒子を用いる流体計算手法は、計算粒子自体が流体を表すため、格子を用いる流体計算手法のように流体の速度の計算の他に流体の界面を追跡する必要がない。さらに、計算格子を用いないため細かい飛沫なども容易に計算することができる。しかし格子を用いた解法ではそれぞれの格子の接続関係が決まっているが、粒子法では計算粒子同士が接続関係を持たず、動的に配置を変更するため、ある座標での物理量を計算するた

めには各タイムステップにおいて、その座標の近傍に存在する粒子を探索しなければならない。計算領域を内包するように計算格子を用意する。そして近傍粒子探索を効率的に行なうために、計算領域に格子を配置して、ある粒子の番号をその粒子が入っているボクセルに格納することで効率化することができる。しかし固定された等間隔格子を用いると、流体の計算領域内の分布が変化するときには粒子が存在しないボクセルが多くなってしまい、メモリ効率が悪い。プログラムが使用可能なメモリの大きさは限りがあるため、格子に用いるメモリを少なくすればするほど、流体に用いることのできるメモリが増え、より大規模なシミュレーションを行なうことができるようになる。またゲームやバーチャル外科手術などでは、ユーザーの視点が移動することがあり、広い領域を計算する必要がある。

本研究では粒子法シミュレーションの近傍探索に用いる格子のメモリ効率の良いデータ構造を提案する。本研究で提案するスライスデータ構造と既存のデータ構造との比較は3章で考察するが、本手法はデータ構造構築が容易であり、データ構造の構築のコストも低いため、リアルタイムアプリケーションにも用いるこ

* 原稿受付 2007 年 7 月 26 日, 改訂年月日 2007 年 9 月 3 日, 発行年月日 2007 年 10 月 4 日, ©2007 年 日本計算工学会。

Manuscript received, July 26, 2007; final revision, September 3, 2007; published, October 4, 2007. Copyright ©2007 by the Japan Society for Computational Engineering and Science.

とができる。また本手法はGPUを用いて高速化することが可能である。そのため、GPUを用いた粒子法シミュレーションに組み込むことも可能である。

本論文ではまずこのスライスデータ構造について説明し、GPUを用いたスライスデータ構造の構築手法を述べる。そしてGPUを用いた粒子法シミュレーションに組み込み、計算時間や必要なメモリの大きさを計測し、本手法の有用性の評価を行なう。

2. 関連研究

粒子法を用いた流体シミュレーション手法はMoving Particle Semi-implicit(MPS) Method⁽²¹⁾とSmoothed Particle Hydrodynamics(SPH)⁽²⁴⁾がある。PremezeらはMPSをコンピュータグラフィックスの分野に導入した⁽³⁰⁾。MPSは連続の式から導出するポワソン方程式を解いて非圧縮流れを計算することができるが、SPHに比べて計算コストは高い。そのためコンピュータグラフィックスの分野では全て陽的に解くことができるSPHが多く用いられている。例えばMüllerらはSPHをリアルタイムアプリケーションに用いた⁽²⁷⁾。コンピュータグラフィックスでは広い領域のシミュレーションを効率よく行なうために粒子径を動的に変更する研究も行なわれた⁽¹⁾。粒子法では粒子間には接続情報はなく、自由に移動できる。そのため、毎タイムステップごとに相互作用を行なう近傍粒子を探索しなければならない。全粒子から近傍粒子を探索すると、計算コストは $O(n^2)$ になり、粒子数が増加するに従って計算コストが高くなる。そこでSPHの計算では固定された等間隔格子を用いてボクセルに入っている粒子のリストを作り、効率化する手法が多く用いられている^(25, 5)。粒子の影響半径が一定の計算ではこのような単一の大きさのボクセルを用いる手法が効率的であるが、それぞれの粒子の影響半径が異なる場合には、Octreeなどの階層格子が用いられる⁽¹⁸⁾。しかし階層格子でのリーフノードのアクセスの計算コストは $O(\log n)$ であるため、粒子の影響半径が同じ場合には、等間隔格子の方が計算効率は良い。

本研究のもう1つの重要な背景はGPUの汎用利用の研究である。GPUはグラフィックスの処理に特化するため、内部はプロセッサが並列に処理をするという構造になっている。GPUは頂点の座標変換などの処理ならばCPUをはるかに超える速度で行なうことができる。GPUはグラフィックス専用のプロセッサであったが、シェーダと呼ばれるプログラムを書くことによって処理を制御することができるようになり、グラフィックス以外の処理に用いることができる可能性が出てきた⁽⁸⁾。そこでGPUを汎用的に利用する研究が多く行なわれている^(28, 29, 7)。セルオートマトンや⁽¹⁶⁾、格子を用いた流体の計算^(15, 17)、布のシミュレーション⁽⁶⁾、ボクセル化⁽⁹⁾などである。また粒子法をGPUで高速化する計算も行なわれてきた。Kipferらは完全な粒子間の衝突を計算しないシミュレーションをGPU上

で行なった⁽¹⁹⁾。またSPHのシミュレーションに関してはAmadaらは近傍粒子探索以外の処理をGPUを用いて高速化した⁽²⁾。Kolbらは物理量を一度格子に分配する手法を研究したが、この手法では数値拡散が起ってしまう⁽²⁰⁾。これらの点を克服しHaradaらはSPHの全てのシミュレーションをGPUを用いて行なう手法を開発した^(11, 10)。しかしこの研究では近傍粒子探索に空間上に固定された等間隔格子を用いるため、メモリ効率が悪く、計算領域に制限が生じる。

等間隔格子の応用手法として必要なボクセルのみのメモリを確保するために、まず仮想的に等間隔格子を考え、それぞれの粒子に対してその粒子が存在しているボクセルの番号を計算する。そしてそのボクセル番号をキーとしてソートして近傍探索を効率的にするデータ構造を構築する手法もある。この手法ではデータ構造構築のために全粒子のソートが必要であり、さらにボクセルに格納されている値にアクセスするために、二分探索を行わなければならないため、頻繁にデータ構造の再構築を行ないボクセルのメモリを読み出す必要のある粒子法シミュレーションには不向きである⁽³¹⁾。

近傍探索にハッシュ格子を用いる方法も存在する⁽³³⁾。1つ1つの格子の座標からハッシュを計算し、3次元の等間隔格子がある長さの1次元の格子に変換する。等間隔格子を用いた場合と比べると粒子が格納されていないボクセルのメモリを確保しなくてもよいという利点がある。しかしこの手法では粒子番号を格納するのに固定の大きさの1次元配列しか必要としないが、1次元配列の1要素に必ず3次元空間内の1個の格子が割り当てられるとは限らない。つまり1要素に複数個の格子が割り当てられることがある。これをハッシュの衝突という。従って、ハッシュを用いて格納されたデータへのアクセスは固定長の処理になるとは限らない。このような処理はGPUで計算するには非効率であるため、固定長の処理でデータアクセスできるハッシュの研究も行なわれている⁽³²⁾。しかしこのデータ構造の構築は計算コストが高く、粒子法シミュレーションのように毎タイムステップにおいてデータ構造を構築しなければならないようなアプリケーションには向いていない。コンピュータグラフィックスの分野ではこのように様々な格子のデータ構造についての研究が存在するが、計算力学分野では格子のデータ構造についての研究は行なわれていない。本論文で提案するスライスデータ構造は今まで研究されていない。

3. 粒子法シミュレーションに求められる条件

近傍粒子探索を効率化するために導入することのできるデータ構造は、固定された等間隔格子、ハッシュ格子、階層格子に分類される。粒子法の近傍粒子探索を行なう上で、必要な条件は3つある。1つ目はデータ構造の構築の計算コストが低いことである。これは粒

子法シミュレーションでは毎タイムステップで粒子が移動するため、このデータ構造を再構築しなければならないからである。2つ目はある座標を含んだボックスに容易にアクセスできることである。これは1タイムステップにおいて多くの近傍粒子を探索する必要があるためこのコストが低いことが望ましい。特に流体シミュレーションでは多くの近傍粒子を探索しなくてはならないので、この計算コストは重要になってくる。そして3つ目はデータ構造に用いるメモリが少ないことである。それぞれのデータ構造に対して、これらの条件を満たしているかを順に考察していく。

まず固定された等間隔格子ではデータ構造の構築の計算コストは粒子数を n とすると $O(n)$ であり、十分に低い。そして2つ目の条件も座標から数回の演算でその点を内包するボックスの番号を求めることができ、その計算コストは粒子数には関係ない。これらの2つの条件は良く満たしているが、計算領域のバウンディングボックス内のメモリを確保するため、粒子を格納しないボックスのメモリも確保する必要があるため、多量のメモリを必要とする。よってメモリ効率は悪い。

ハッシュ格子は全てのボックスがメモリの1要素に割り当てられれば、ボックスへのアクセスが容易であり、多量のメモリも必要としないため、望ましいデータ構造である。しかし一般的にいくつかのボックスがメモリの1要素に割り当てられるハッシュの衝突が起こるため、ボックスへのアクセスはハッシュの衝突の回数によって変わってしまう。よってデータ構築とボックスへのアクセスのコストはハッシュの衝突が多くなるほど非効率になる。

階層格子はデータ構造の構築には $O(n \log n)$ のコストがかかるため、等間隔格子と比べるとコストは高い。階層構造にすることで複数の階層のボックスのメモリを確保する必要があるが、それらの数は一般的に最下位の不要なボックスの数に比べると少ないため、階層格子のメモリ効率は良い。またボックスへのアクセスは等間隔格子のように座標からその座標を含んだボックスの番号を直接計算することはできず、階層構造をたどって行かなければならず、 $O(\log n)$ の計算を行わなければならない。この $\log n$ の計算コストは少数の粒子の計算では十分に小さいが、本研究で取り扱うような多数の粒子が存在する広い領域での計算においては大きくなる。

またアルゴリズムをGPUのような複数の計算ユニットが並列で処理を行なうプロセッサ上で実行するときには、処理を全て並列化することができることが望まれる。これはCPUで処理を行なう必要があると、それらの間で速度の遅いデータ転送を行わなければならないためである。またもう1つの条件としては全ての計算ユニットでの負荷がほぼ均等になっていることが望ましい。

本論文では流体や粉体などの粒子法シミュレーションに適したデータ構造を提案することであり、これら

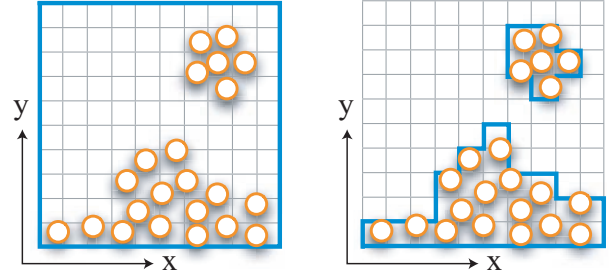


Fig.1 A fixed grid and a dynamic grid constructed by the present method. Memories are allocated in the region inside of the blue lines.

のシミュレーションでは全ての粒子の大きさは一定のものが一般的であり、このような場合には単位体積あたりの粒子数がほぼ一定になる。つまり、粒子の疎密が生じないということであり、粒子が存在している部分では空間を等分割したボックスに割り当てられる粒子数はほぼ一定となる。等間隔格子は、Harada らが示したように全てを並列化することができ⁽¹¹⁾、さらにボックスへのアクセスの計算コストは全て同じであるため、計算負荷のバランスもよい。ハッシュ格子はハッシュの衝突によりこれらの条件を満たすのが困難である。まずハッシュの衝突は毎タイムステップ異なるので、データを格納するために、メモリ構造を再構築しなければならないだけでなく、この処理をGPUで行なった研究はない。またボックスへのアクセスもハッシュの衝突により、計算の長さが変わってしまうため、計算ユニットでの負荷は均一にはなるとは限らない。

これらをまとめると等間隔格子はメモリ効率が悪く、ハッシュ格子はGPU上での実装に向かない。また階層格子はデータ構造の構築とボックスへのアクセスの計算コストが高い。本論文で提案するスライスデータ構造は、前述の条件を全て満たしている。データ構造の構築の計算コストは $O(n)$ であり、ボックスへのアクセスの計算コストも n に依存せず、等間隔格子を用いた場合とほぼ同じである。また大部分の不要なボックスのメモリを確保しないため、データ構造に必要なメモリも小さい。さらに全ての処理をGPU上で行なうことができ、計算負荷に偏りが無い。

4. スライスデータ構造

本手法ではまず計算に用いられる空間に等間隔格子を用意する。しかしここでは仮想的に考えるだけであり、実際にメモリの確保はしない。この格子を構成する1つ1つの立方体をボックスと呼ぶ。この格子は無限の広がりを持つものであり、計算中は空間上の配置を変更しない。そして本手法ではこれらのボックスのうち必要なものをメモリ上に保持する。等間隔格子を用いる場合だと計算領域のバウンディングボックスを定め、その内部のボックスのためのメモリを確保していた。

本手法は計算空間に用意されたボクセルの1軸に直交するスライスに格子を分割する。スライスの次元は計算領域の次元よりも1次元低いものになる。この分割ではそれぞれのスライスが軸方向に1ボクセルだけ持つようにする。計算領域が3次元であり、Y軸方向を軸と定めるとスライスはXZ軸方向に広がりを持つ2次元格子になる。以下ではこのように3次元の計算領域をY軸方向に垂直なスライスに分割した場合を例として説明していく。計算領域をスライスに分解した後、それぞれのスライスにおいてXZ軸方向のバウンディングボックスを定義する。つまりXZ軸方向における最大と最小のボクセルの開始座標 $b_{i,max}^x, b_{i,min}^x, b_{i,max}^z, b_{i,min}^z$ を計算する。ここで i はスライスの番号である。するとスライス i 上のXZ方向のボクセルの数 n_i^x, n_i^z は以下のように計算できる。

$$n_i^x = \frac{b_{i,max}^x - b_{i,min}^x}{d} + 1 \quad (1)$$

$$n_i^z = \frac{b_{i,max}^z - b_{i,min}^z}{d} + 1 \quad (2)$$

d はボクセルの1辺の長さである。これらを用いてスライス i 上でのボクセルの数を $n_i = n_i^x n_i^z$ と求めることができる。本手法ではスライス内のバウンディングボックス内の領域のみ、メモリを確保する。

ここで計算領域が n 個のスライス $\{S_0, S_1, S_2, \dots, S_{n-1}\}$ に分解されるとする。そしてスライス S_i での先頭のボクセルの番号 p_i はスライス S_0 から S_{i-1} にあるボクセルの数の和として

$$p_i = \sum_{j<i} n_j \quad (3)$$

と計算する。するとある点 x, y, z が存在するボクセルの番号を計算するために、まずY軸方向の最小のボクセルの開始座標 b_{min}^y を用いて、その点が含まれるスライスの番号を以下のように計算する。

$$i = \left\lceil \frac{y - b_{min}^y}{d} \right\rceil \quad (4)$$

そしてボクセル番号 $v(x, y, z)$ は

$$v(x, y, z) = p_i + \left(\left\lceil \frac{x - b_{i,min}^x}{d} \right\rceil + \left\lceil \frac{z - b_{i,min}^z}{d} \right\rceil n_i^x \right) \quad (5)$$

と計算することができる。式(5)より、ある点が存在するボクセル番号を計算するために必要な値は各スライスのバウンディングボックスを定義する値 $b_{i,min}^x, b_{i,min}^z, n_i^x$, スライス番号を決定する値 b_{min}^y , 各スライスの先頭番号 p_i , そしてボクセルの一辺の長さ d であることがわかる。このデータ構造を用いると図1右に示すようにメモリ効率よく格子を作成することができる。このデータ構造をスライスベースデータ構造と呼ぶ。

5. GPUを用いた実装

スライスベースデータ構造を構築するにはまずスライスのバウンディングボックスを決定し、スライスの先頭番号をそれらから計算する。このようにしてボク

セルの番号を計算する値を求めた後、ボクセルに値を書き込んで行く。この値は粒子の近傍探索を行なう場合には粒子番号であるが、等値面を構築するときに濃度を計算するのならば濃度値である。

本章ではまず、GPU上で実装するためのデータ構造について述べ、それから処理を順番に述べる。また以下ではボクセルに粒子番号を格納する場合を例にあげ説明する。GPUを汎用利用する基礎についてはここでは説明しない。基礎的な技術に関しては参考文献に譲る^(13, 14)。

5.1 データ構造 GPU上で計算を行なうためには、データをテクスチャとしてビデオメモリに格納しなければならない。まず各スライスに必要な値を格納するために、1次元テクスチャを用意する。スライスの数が1次元テクスチャの大きさの最大値を超えるならば2次元テクスチャにすることもできる。各スライスに必要な値は $p_i, b_{i,min}^x, b_{i,min}^z, n_i^x$ の4個であるため、1次元テクスチャのRGBAチャンネルにこれらを格納すれば1枚のテクスチャで十分である。また格子に保持する値を格納するための2次元テクスチャを用意する。このテクスチャをプールテクスチャと呼ぶ。ここでは粒子番号をプールテクスチャに格納することを例として説明する。本研究ではHaradaらの研究で行なわれたように、1個のボクセルに1テクセルを割り当て、最大4個の粒子番号を格納するようにした⁽¹¹⁾。この場合では2次元テクスチャのテクセル数がプールテクスチャに格納できるボクセルの最大値である。

5.2 スライスのバウンディングボックスの決定 スライスでのバウンディングボックスを決定するためにまず各スライスでの最大と最小のボクセルの開始座標を計算する。この処理は粒子のY座標からスライスの番号を計算し、各スライスに存在する粒子座標の最大値と最小値を計算することによって行なうことができる。GPUを用いる場合は粒子1個に頂点を1個割り当て用意した1次元テクスチャに書き込む。パーティックスシェーダでは粒子のY座標から式(4)でスライス番号、つまり1次元テクスチャ上に書き込む位置を計算し、その位置に頂点を出力する、そしてフラグメントシェーダでXZ座標を色として出力する。最大値最小値の選択はアルファブレンディング*を用いて行なうことができる⁽²³⁾。

5.3 先頭番号の計算 各スライスでのXZ座標の最大値と最小値を用いることでそのスライス上のボクセルの数を計算することができる。先頭番号の計算は各スライスにおいて式(3)を計算しなければならない。この処理をスライス i に対して n_0 から n_{i-1} までの値を読み出して計算することになる。スライスの総

*アルファブレンディングとは指定されたアルファ値を用いて半透明な色を重ね合わせて描画することである。アルファ値を用いてバッファの色と描画する色をどのように混合させるかを指定できる。機能の中に色の最大値を描画するようにする機能があり、それをここでは使用した。

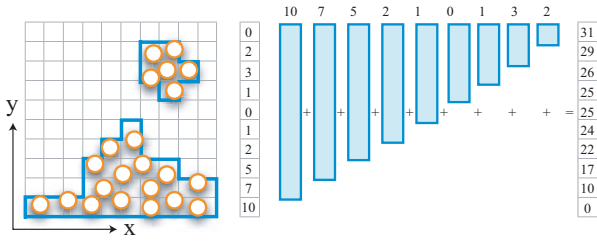


Fig.2 Computation of leading values of slices.

数が m だとすると最も多くメモリアクセスを行わなければならない計算ユニットは $(m(m+1))/2$ 回メモリアクセスが必要になり、メモリ転送速度に処理の速度が制限される。GPUを用いるとこの処理はフラグメントシェーダを用いた Gathering Operation になる。GPU はメモリバンド幅は広いが、転送速度は遅いため、計算の並列度を高くしてこの点を隠蔽している。そのため、1 個の計算ユニットのメモリアクセス回数を減らし、並列度を高めた方が効率的に処理ができる。行列やベクトルの Reduction が、類似した計算であり 1 つの計算ユニットで多くのメモリアクセスを行なうより、何段階かに処理を分けて、1 個の計算ユニットのメモリアクセスを減らし並列度を高めて行なった方が効率が良いことが知られている。(22, 4).

そこで本手法ではスライス i に存在するボクセル数 n_i を読み出して $i+1$ から m 番目のスライスの先頭番号配列に加算していく。こうすることによって m 回のメモリアクセスで先頭番号を計算することができる。この処理は Scattering Operation であり、GPU を用いる場合はバートックスシェーダを用いて行なうことができる。この処理はスライスが m 個あるとすると要素数 m の 1 次元配列に書き込む処理である。つまり GPU を用いる場合は m テクセルの 1 次元テクスチャに書き込むことになり、スライス i に存在するボクセル数 n_i を $i+1$ から m 番目のスライスの先頭番号配列に書き込む処理は、 $i+1$ 番目のテクセルから m 番目のテクセルまでの線分を描画することで行なうことができる。このときにそのスライス内のボクセルの数を色として出力し、アルファブレンディングで加算処理を行なうことで式 (3) を評価し、各スライスの先頭番号を計算することができる⁽²³⁾。つまり各スライスごとに線分を用意する必要がある。図 2 はこの操作を示したものである。

5.4 値の格納と読み出し このようにして計算したスライスの先頭番号とバウンディングボックスを定義する値を用いて、粒子番号をプールテクスチャに格納していく。粒子の Y 座標からスライス番号を計算して、そのスライスの先頭番号 p_i とスライスでの最小値 $b_{i,min}^x, b_{i,min}^z$ 、スライスでの X 軸方向のボクセル数 n_i^x を読み出す。これらを用いて式 (5) を計算し、その粒子に割り当てられているボクセルのインデックスを計算することができる。計算されたインデックスに対

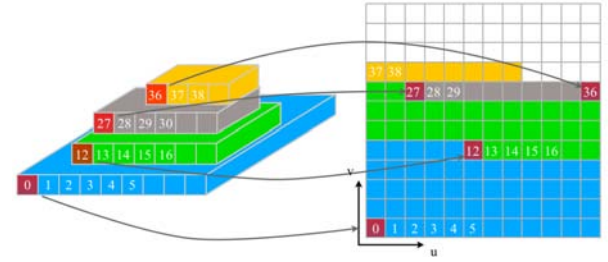


Fig.3 This figure shows that how the voxels are allocated in the texture memory. The voxel is allocated in the memory from the bottom of the slices.

応するテクセルへの値の書き込みは、粒子 1 個に頂点を 1 個割り当てて、そのテクセルに粒子番号を色として書き込むことを行なうことができる。図 3 にスライスのボクセルがどのようにテクスチャに格納されているかを示す。下のスライスのボクセルからテクスチャに格納される。

ある座標が存在するボクセルに格納されている値の読み出しは、その座標から値の格納時と同様にボクセルのインデックス、つまりメモリのアドレスを計算することによって、行なうことができる。

6. 粒子法シミュレーションへの導入

本手法は粒子番号を格納する格子データ構造の構築と、粒子番号の格納まで GPU で行なうことができる。そのため全て GPU 上で行なうことができるシミュレーションに組み込むことができる。GPU を用いた粒子法の研究は Distinct Element Method (DEM)⁽¹²⁾ や SPH⁽¹¹⁾、粒子ベースの剛体のシミュレーション⁽³⁴⁾ があり、本研究では DEM と SPH のシミュレーションに組み込んだ。

DEM と SPH の GPU での実装の詳細については参考文献で説明されているため、ここでは省略する^(12, 11)。本論文で提案しているデータ構造を組み込むためには、格子に粒子番号を格納する部分と格子から粒子番号を読み出す部分を変更するだけでよい。1 ボクセルに 4 個の粒子番号を格納する処理は、等間隔格子の場合と同様に Harada らが開発した深度テストとステンシルテスト、カラーマスクを用いる手法を用いて行なうことができる⁽¹²⁾。

7. 結果

本手法を Core2 X6800 CPU, GeForce 8800GTX GPU, 2GB のメモリを搭載した PC 上で、プログラムは C++, OpenGL, C for Graphics を用いて開発し、粒子法シミュレーションに組み込んだ^(8, 3)。

DEM の 3 次元シミュレーションの計算例を図 4 に示す。65,536 粒子を用いたシミュレーションであり、立方体の容器内で柱状に配置した粒子を落下させた計算を Z 軸方向に正射影して描画した結果である。レンダリ

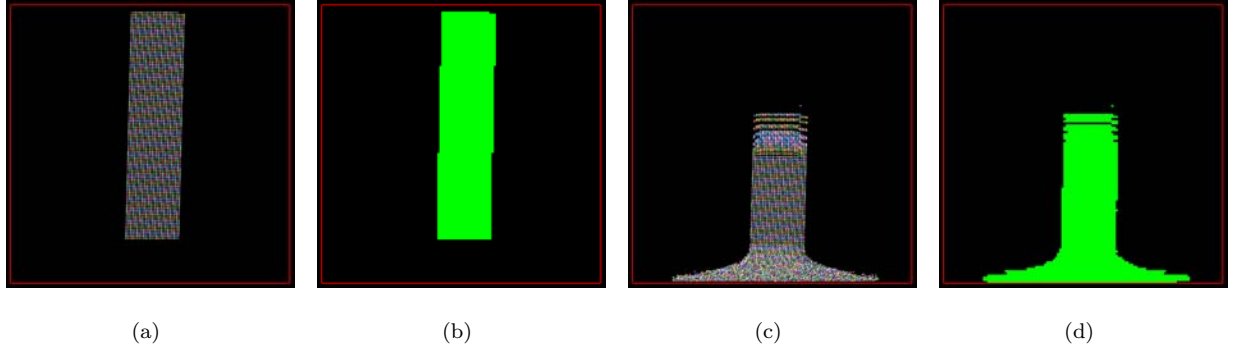


Fig.4 A particle-based simulation using a dynamic grid which is constructed by the present method. The green areas in (b) and (d) indicate the grids allocated in the memories.

ングにはポイントスプライトを用いて粒子位置に球を描画した。本手法を用いずに等間隔格子を用いた場合は赤線で囲まれた領域全体を格子として 128^3 ボクセルのメモリを確保しなければならないが、本手法を用いた場合は緑色で示した領域のみメモリを確保すればよい。等間隔格子を用いた場合は粒子番号を格納するメモリとして 32MB 必要なのに対して、図の計算例では最大約 2MB しか必要なかった。本手法を用いると格子が粒子分布にタイトになり、メモリ効率が良くなっていることがわかる。表 1 は 128^3 のボクセルに必要なメモリ量に対する、本手法を用いた計算において粒子番号を格納するメモリとして確保したメモリの量の割合を示している。本手法を用いると等間隔格子を用いた場合の約数パーセントから数十パーセントのボクセルのみメモリを確保すれば良いことがわかる。しかし粒子数 589,824 の場合は計算領域の約半分の量のボクセルのメモリを確保しなかった。表 2 には DEM のシミュレーションで総粒子数を変えて計算したときの計算速度の比較を示す。この表の格子の作成の時間とはスライスを決める値を計算し、粒子番号をプールに格納するまでの処理である。表より全ての計算例において格子作成の時間は本手法を用いた場合の方が速いことがわかる。しかし、この時間の差は総粒子数が多くなればなるほど小さくなっている。これはまた、本手法を用いた時に用意しなければならないボクセル数が、等間隔格子を用いた場合に近づくほど大きくなっているとも見ることができる。これは本手法ではスライスを決める値を計算する必要があるが、プールのテクスチャの大きさが等間隔格子のテクスチャよりも小さいからであり、テクスチャが大きいほどテクスチャに書き込むときの用意に要するオーバーヘッドが長いことに起因すると考えられる。また総計算時間は本手法を用いた場合の方が速いが格子の作成の時間の差よりも計算時間の差は縮まっている。これは座標からその座標が入っているボクセルに格納されているデータにアクセスするときに等間隔格子よりも多く計算を行わなければならないからであると考えられる。

次に本手法を SPH の 3 次元シミュレーションに用い

た場合の計算時間と等間隔格子を用いた場合の計算時間の比較を表 3 に示す。計算領域の広さは粒子径を 1 とすると $256 \times 256 \times 256$ である。すなわち等間隔格子を用いた場合は 256^3 ボクセル分のメモリを確保しなければならない。この等間隔格子は 256MB のメモリを必要とするのに比べ、本手法を用いた 1,048,576 粒子のシミュレーションでは最大約 15MB のメモリで計算することができた。表から本手法を用いた場合の方が計算時間が短くなっていることがわかる。例えば表 2 の 65,536 粒子を用いた場合の格子作成の計算時間の差と、表 3 の 1 タイムステップの計算時間の差を比較すると 1 タイムステップの計算時間の差の方が大きい。つまり格子生成以外の計算においても本手法を用いた場合の方が計算時間が短いということである。格子生成以外の計算では粒子番号をテクスチャから読み出す部分しか差がない。つまり、粒子番号をテクスチャから読み出す部分において、本手法の方が計算量が多いけれども速度は速いということである。これは本手法を用いた場合の方がキャッシュヒット率が高くなったことが原因であると考えられる。本手法を用いた場合は格子テクスチャに粒子番号がより密に格納されているため、粒子が存在するボクセルの値がより多くキャッシュに残るようになったと考えられる。[†]

図 5 は 2 個の計算例のスクリーンショットとそのタイムステップでのプールのテクスチャを示したものである。流体シミュレーション結果のレンダリングには DEM と同様にポイントスプライトを用いたが、1 次元テクスチャから密度の値を用いて色を計算した。密度が低いものは白くレンダリングされている。プールのテクスチャの黒いテクセルは粒子番号が格納されていないボクセルであり、赤、黄、白のテクセルはそれぞれ 1, 2, 3 個のインデックスが格納されているものである。計算領域は Y 軸に垂直な面でスライスされており、スライスは図 3 に示すように計算領域の下からプールテクスチャに格納されている。プールテクスチャの縦

[†]GPU のキャッシュサイズについては明らかにされていない。また GPU のキャッシュヒット率は低いと構造に関しては CPU とほぼ同じ構造である (26)。

横はテクスチャ座標の u, v の軸である．よってテクスチャの上部は計算領域の上部のボクセルを格納している．図上の計算例は本手法が特に効果的であるシーンである．プールのテクスチャを見ても密にデータが格納されている．図下の例でも比較的密にデータが格納されているが、テクスチャ上部に空のボクセルが見られる．これらは液滴によって水面が乱されている部分によるものである．

粒子法のシミュレーションはGPUで行なった方が高速であり、原田らの研究では最大16倍の高速化が行なわれた⁽¹²⁾．粒子の番号を格子に格納する計算のCPUとGPUでの速度の比較は行なわれていないが、1タイムステップの計算での計算時間の差からこの処理もGPUで行なった方が十分高速であると推測される．本研究で提案するスライスデータ構造を用いた粒子番号の格子への格納をCPUとGPUで行ない、計算時間を測定して比較したものを図7に示す．この結果よりGPUでの計算の方が約5倍から約45倍高速に計算できている．

3章で述べたように階層格子はボクセルへのアクセスの計算コストは高いがメモリ効率は良い．そこでスライスデータ構造と階層格子の1つであるOctreeのメモリ消費量を比較した結果を図8に示す．この結果は図4のシミュレーションにおけるメモリ消費量である．計算開始直後は粒子は直方体内に整列しており、この時にはスライスデータ構造はメモリを無駄にすることがなく、確保するメモリ量はOctreeの葉ノードのメモリ量と等しい．しかしOctreeでは階層構造を構築しなければならないため、スライスデータ構造よりも多くのメモリを必要とする．計算が進むにつれ、スライスデータ構造では無駄なボクセルのメモリも確保してしまうため、Octreeよりも多くのメモリを使用するという結果になった．しかし図8の最後の状態での等間隔格子のメモリ消費量に対するスライスデータ構造とOctreeのメモリ消費量はそれぞれ6.25%、3.24%であったため、スライスデータ構造は階層格子ほどではないが、効果的にメモリ消費量の削減ができていたことがわかる．

また本手法は等間隔格子を用いないため、計算領域の大きさの制限を受けない．そのため、広い領域であっても容易に計算することが可能である．図6に広い領域内で計算を行なった例を示す．この計算は約4,000,000粒子を用いた計算結果であり、計算領域の広さは粒子径を1とすると $700 \times 700 \times 256$ である．つまり等間隔格子を用いると、粒子番号を格納する格子だけで約2GBのメモリが必要になる．このメモリの大きさは現在利用できるグラフィックスカードの最大メモリ量を超えているため、等間隔格子を用いたのでは不可能な領域の大きさの計算である．しかし本手法を用いた場合は最大約150MBのメモリで計算することができた．

本手法では1次元テクスチャの最後の1テクセルの1チャンネルの値を読み出すことでプールに必要な最大粒

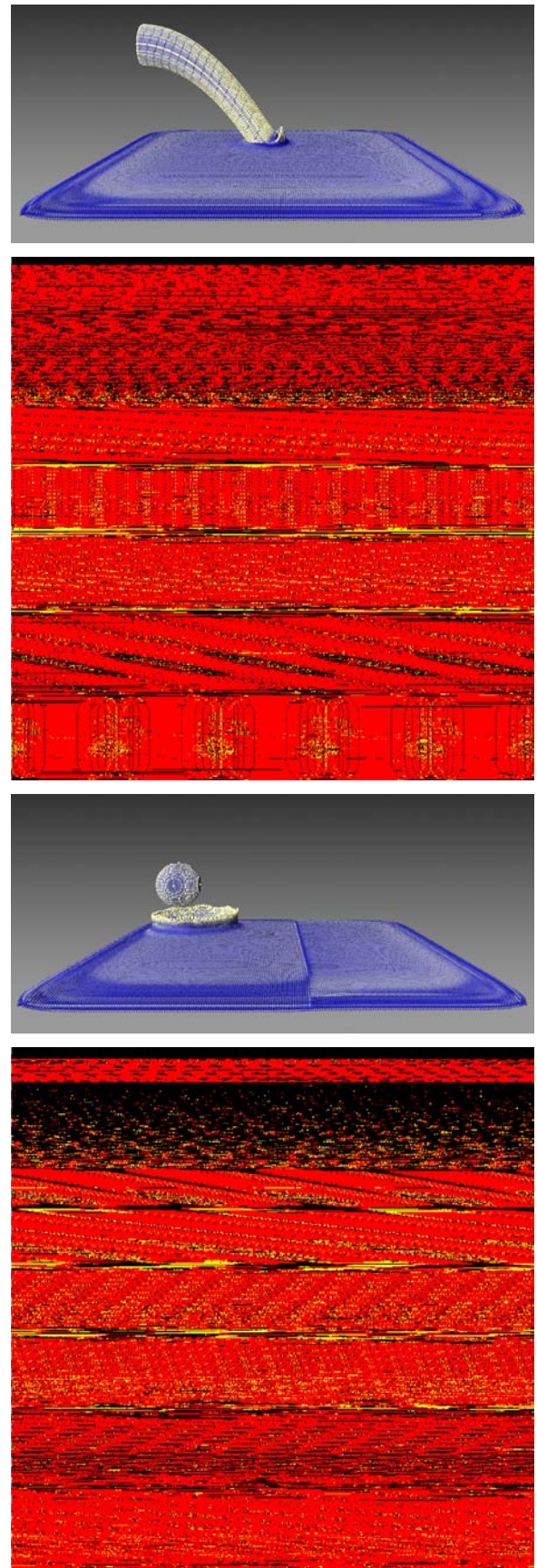


Fig.5 Simulation results and index pool textures. Black texels in the textures indicate empty voxels. There are 1, 2 and 3 indices stored in red, yellow and white texels, respectively.

Table 1 Ratio of the maximum number of voxels used in the present method and total number of voxels in the computational region.

Number of particles	Ratio of voxels
16,386	0.0394
65,536	0.0619
262,144	0.269
589,824	0.488

Table 2 Computation times of DEM simulations in 128^3 grid (in milliseconds). Computation time for construction of a fixed grid is shown in (a) and computation time for construction of a grid using the present method is shown in (b). Total computation time using a fixed grid is shown in (c) and those using the present method is shown in (d). The maximum number of voxels used in the present method is shown in (e).

Number of particles	(a)	(b)	(c)	(d)	(e)
16,386	1.48	0.688	2.53	2.19	82,675
65,536	3.91	2.56	9.68	8.58	129,781
262,144	13.9	10.3	42.0	36.9	564,354
589.824	31.7	23.0	96.4	90.9	1,022,551

子数がわかる。少ないデータ量の通信で最大粒子数がわかるという点も本手法の利点の1つである。これを用いると容易にプールの大きさを変えることも可能である。毎タイムステップ最大粒子数をCPUに読み戻して動的に必要な大きさのプールテクスチャを用意するプログラムも開発したが、計算時間を測定すると固定の大きさのプールテクスチャを用いた場合の方が高速であった。これはメモリの解放と確保に時間がかかったためであると考えられる。そのため、議論してきた計算速度はプールの大きさは動的に変化させず固定の大きさにしたプログラムを用いたものである。本手法を用いると前のタイムステップで用いたプールのメモリの大きさと、次のタイムステップで用いたプールの大きさは前述のように容易に比較することができる。よって毎タイムステップでプールの大きさを変更しなくとも、確保しているプールの大きさよりも大きなメモリが必要になったときにメモリを余裕を持って大きくすることによって、メモリ解放と確保の回数を減らして動的にプールの大きさを変更することができる。

8. 結論

本論文では粒子法シミュレーションの近傍探索に有用なスライスデータ構造を提案した。このデータ構造を用いることで粒子配置にタイトに計算格子を作成することができるようになった。スライスデータ構造ではボクセルのアクセスは固定された等間隔格子とほぼ同じ計算コストであり、階層格子に近いメモリ削減を

Table 3 Computation times of SPH simulations in 256^3 grid (in milliseconds). Total computation times using a fixed grids are shown in (a) and those using the present method are shown in (b).

Number of particles	(a)	(b)
65,536	60.9	53.1
262,144	280.5	231.3
589.824	685.9	567.9
1,048,576	1160.9	1070.3

Table 4 Computation times for construction of the sliced data structure on the GPU (a) and on the CPU (b)

Number of particles	(a)	(b)
16,386	0.688	31.2
65,536	2.56	37.4
262,144	10.3	68.7
589.824	23.0	117.2

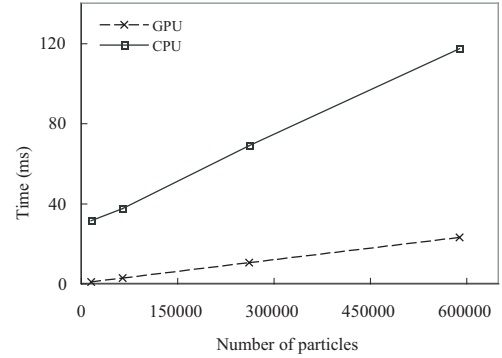


Fig. 7 Comparison of construction times of the sliced data structure on the GPU and on the CPU.

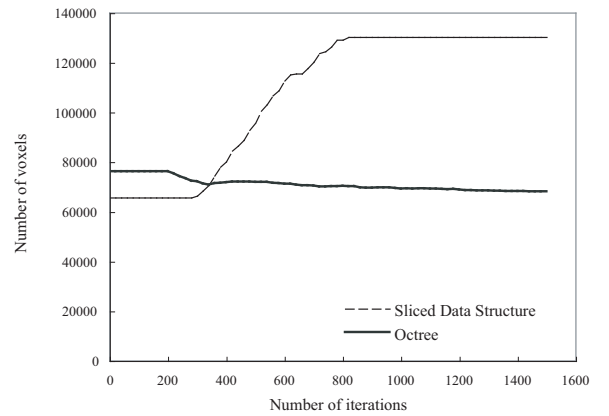


Fig.8 Comparison of the number of voxels.

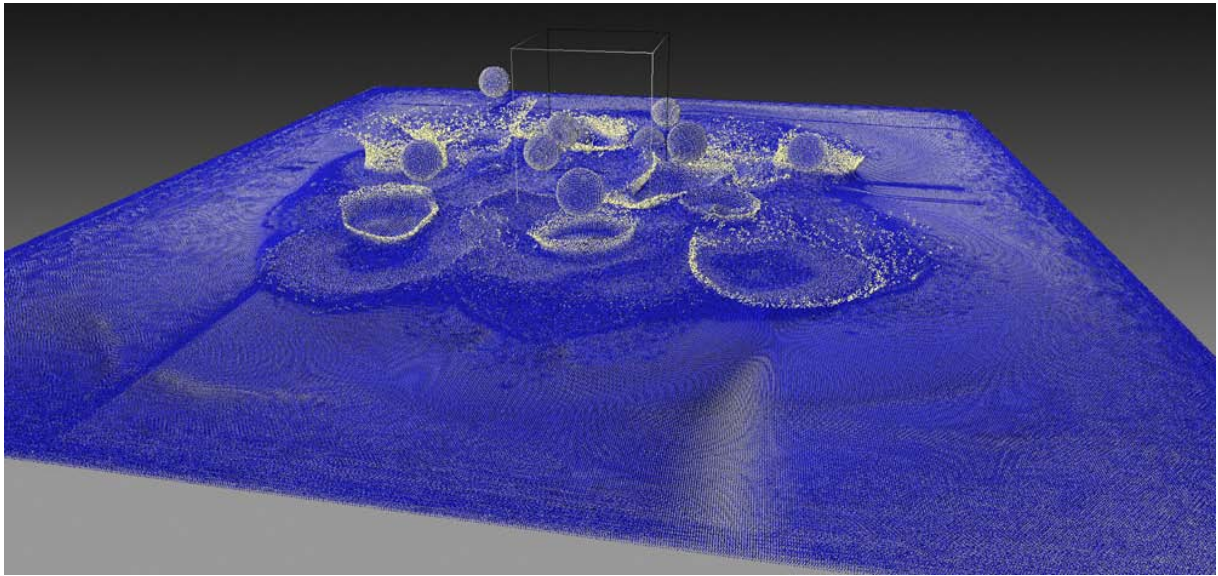


Fig.6 A simulation of a large area.

実現した。またこのデータ構造の構築等をGPUを用いて実装し、GPUを用いたDEMとSPHの粒子法シミュレーションに導入し、手法の定量的評価を行なった。

本手法を用いることによって格子に用いるメモリの量が減り、メモリ効率が上がっただけでなく、SPHの流体計算においては計算時間の向上も見られた。本論文で提案手法の有用性を示すことができた。

参考文献

- (1) B. Adams, M. Pauly, R. Keiser, and L.J. Guibas. Adaptively sampled particle fluids. *ACM Transactions on Graphics*, Vol. 26, No. 3, 2007.
- (2) T. Amada, M. Imura, Y. Yasumoto, Y. Yamabe, and K. Chihara. Particle-based fluid simulation on gpu. In *2004 ACM Workshop on General-Purpose Computing on Graphics Processors*, 2004.
- (3) OpenGL Architecture Review Board, D. Shreiner, M. Woo, J. Neider, and T. Davis. *OpenGL Programming Guide 5 edition*. Addison-Wesley Professional, 2005.
- (4) I. Buck. *GPU Gems2*, chapter Taking the Plunge into GPU Computation. Addison-Wesley Pearson Education, 2005.
- (5) E. R. Clifford. A fast algorithm for calculating particle interactions in smooth particle hydrodynamics simulation. *Computer Physics Communications*, Vol. 70, pp. 478–482, 1992.
- (6) Z. Cyril. Cloth simulation on the GPU. In *ACM SIGGRAPH Sketches, No.39*, 2005.
- (7) R. Fernando. *GPU gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*. Addison-Wesley Professional, 2004.
- (8) R. Fernando and M. Kilgard. *The Cg Tutorial*. Addison-Wesley Pearson Education, 2003.
- (9) T. Harada and S. Koshizuka. Fast solid voxelization using graphics hardware. *Transactions of Japan Society for Computational Engineering and Science, No.20060023*, 2006.
- (10) T. Harada, S. Koshizuka, and Y. Kawaguchi. Real-time fluid simulation coupled with cloth. In *Proc. of Theory and Practice of Computer Graphics*, 2007.
- (11) T. Harada, S. Koshizuka, and Y. Kawaguchi. Smoothed particle hydrodynamics on GPUs. In *Proc. of Computer Graphics International*, pp. 63–70, 2007.
- (12) T. Harada, M. Tanaka, S. Koshizuka, and Y. Kawaguchi. Acceleration of distinct element method using graphics hardware. *Transactions of JSCES*, Vol. 20070011, , 2007.
- (13) M. Harris. *GPU Gems1*, chapter Fast Fluid Dynamics Simulation on the GPU. Addison-Wesley Pearson Education, 2004.
- (14) M. Harris. *GPU Gems2*, chapter Mapping Computational Concepts to GPUs. Addison-Wesley Pearson Education, 2005.
- (15) M.J. Harris, W.V. Baxter, T. Scheuermann, and A. Lastra. Simulation of cloud dynamics on graphics hardware. In *Proc. of the SIGGRAPH / Eurographics Workshop on Graphics Hardware*, pp. 92–101, 2003.
- (16) M.J. Harris, G. Coombe, T. Scheuermann, and A. Lastra. Physically-based visual simulation on

- graphics hardware. *Proc. of the SIGGRAPH / Eurographics Workshop on Graphics Hardware*, pp. 109–118, 2002.
- (17) M.J. Harris, G. Coombe, T. Scheuermann, and A. Lastra. Real-time 3d fluid simulation on GPU with complex obstacles. In *Proc. of the Computer Graphics and Applications, 12th Pacific Conference on (PG'04)*, pp. 247–256, 2004.
- (18) L. Hernquist and N. Katz. Treesph: A unification of sph with the hierarchical tree method. *The Astrophysical Journal Supplement Series*, Vol. 70, pp. 419–446, 1989.
- (19) P. Kipfer, M. Segal, and R. Westermann. Uberflow: A GPU-based particle engine. *Proc. of the ACM SIGGRAPH/EUROGRAPHICS Conference on Graphics Hardware*, pp. 115–122, 2004.
- (20) A. Kolb and N. Cuntz. Dynamic particle coupling for GPU-based fluid simulation. In *Proc. of 18th Symposium on Simulation Technique*, pp. 722–727, 2005.
- (21) S. Koshizuka and Y. Oka. Moving-particle semi-implicit method for fragmentation of incompressible fluid. *Nucl.Sci.Eng.*, Vol. 123, pp. 421–434, 1996.
- (22) J. Krüger and R. Westermann. Linear algebra operators for GPU implementation of numerical algorithms. *ACM Transactions on Graphics*, Vol. 22, No. 3, pp. 908–916, 2003.
- (23) E. Lengyel. *The OpenGL Extensions Guide*. Charles River Media, 2003.
- (24) J.J. Monaghan. Smoothed particle hydrodynamics. *Annu.Rev.Astrophys.*, Vol. 30, pp. 543–574, 1992.
- (25) J.J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics*, Vol. 30, pp. 543–574, 1992.
- (26) J. Montrym and H. Moreton. The geforce 6800. *IEEE Micro*, Vol. 25, No. 2, pp. 41–51, 2005.
- (27) M. Müller, D. Charypar, and M. Gross. Particle-based fluid simulation for interactive applications. In *Proc. of SIGGRAPH Symposium on Computer Animation*, pp. 154–159, 2003.
- (28) John D. Owens, David Luebke, Naga Govindaraju, Mark Harris, Jens Krüger, Aaron E. Lefohn, and Timothy J. Purcell. A survey of general-purpose computation on graphics hardware. *Eurographics 2005, State of the Art Reports*, pp. 21–51, 2005.
- (29) M. Pharr and R. Fernando. *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*. Addison-Wesley Professional, 2005.
- (30) S. Premoze, T. Tasdizen, J. Bigler, A. Lefohn, and R.T. Whitaker. Particle-based simulation of fluids. *Computer Graphics Forum*, Vol. 22, No. 3, pp. 401–410, 2003.
- (31) T.J. Purcell, M. Cammarano, H.W. Jensen, and P. Hanrahan. Photon mapping on programmable graphics hardware. *Proc. of the ACM SIGGRAPH/EUROGRAPHICS Conference on Graphics Hardware*, pp. 41–50, 2003.
- (32) L. Sylvain and H. Huges. Perfect spatial hashing. *ACM Transactions on Graphics*, Vol. 25, No. 3, 2006.
- (33) M. Teschner, B. Heidelberger, M. Müller, D. Pomeranets, and M. Gross. Optimized spatial hashing for collision detection of deformable objects. In *Proc. of Vision, Modeling, Visualization*, pp. 47–54, 2003.
- (34) 原田隆宏, 田中正幸, 越塚誠一, 河口洋一郎. GPUを用いたリアルタイム剛体シミュレーション. 情報処理学会研究報告, No.126, pp. 79–84, 2007.